LABORATORY OBSERVATION

Course:	Full Stack Web Development	**Experiment No.:**	6 (Exp 4)
Course Code:	23CM4121	**Date:**	05-10-2026
Student Name:	Nimidithalli Nandu	**Roll No.:**	A24126552100

1. TITLE

Implementation of Application-Level Logging Middleware in Express.js

2. AIM

To build an Express.js server incorporating custom application-level middleware for request logging, timestamp recording, and HTTP route handling.

3. OBJECTIVE

- Understand the middleware lifecycle and the role of next() in Express.
- Implement a custom logger middleware capturing request timestamp, method, and URL.
- Register middleware globally across multiple endpoint routes (/ and /about).
- Verify asynchronous request interception and execution flow.

4. THEORY

Middleware functions in Express are functions that have access to the request object (req), response object (res), and the next middleware function in the application's request-response cycle.

Common middleware duties include executing code, altering request and response objects, ending request-response cycles, and calling the next middleware using next().

app.use(logger) registers an application-level middleware that runs for every incoming HTTP request regardless of path or method.

5. ALGORITHM / PROCEDURE

1. Initialize an Express application instance using express().

2. Declare a custom middleware function logger(req, res, next).

3. Inside logger, print the current locale timestamp, HTTP method, and requested URL.

4. Invoke next() to yield control to subsequent route handlers.

5. Register the logger globally using app.use(logger).

6. Define GET routes for "/" (Welcome greeting) and "/about" (About page).

7. Start the server on port 3000 using app.listen(3000).

6. PROGRAM

package.json

```
{
  "name": "middleware",
  "version": "1.0.0",
  "description": "Express middleware implementation with custom logger",
  "main": "code.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node code.js"
  },
  "keywords": [
    "middleware",
    "express",
    "logger"
  ],
  "author": "Nimidithalli Nandu",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

code.js

```
const express = require("express");

const app = express();


// Logging middleware
function logger(req, res, next) {

    console.log(
        new Date().toLocaleString(),
        req.method,
        req.url
    );

    next();
}


// Use middleware
app.use(logger);


// Routes
app.get("/", (req, res) => {
    res.send("Welcome to my server");
});

app.get("/about", (req, res) => {
    res.send("This is the about page");
});


app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```

7. CODE EXPLANATION

Code Element	Detailed Explanation
function logger(req, res, next)	Defines custom middleware receiving request, response, and next callback.
req.method, req.url	Accesses incoming request HTTP method (GET, POST) and endpoint path.
next()	Hands off execution to the next middleware or matching route handler.
app.use(logger)	Registers the logging middleware globally on all incoming application requests.
app.get("/about", ...)	Handles GET requests to the /about endpoint after passing through middleware.

8. TEST CASES AND EXECUTION DATA

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Initialize npm package	Create package.json via npm init -y	package.json created successfully	**Pass**
TC-02	Install express module	Install express and generate lockfile	Express installed and verified	**Pass**
TC-03	Launch server with logger	Server listens and activates middleware	Server running on port 3000	**Pass**

9. OUTPUT

Terminal Initialization and Server Execution of middleware/code.js

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task6\middleware> npm.cmd init -y
Wrote to C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task6\middleware\package.json:

{
  "name": "middleware",
  "version": "1.0.0",
  "description": "Express middleware implementation with custom logger",
  "main": "code.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "Nimidithalli Nandu",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task6\middleware> npm.cmd install express

added 68 packages, and audited 69 packages in 3s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task6\middleware> node code.js
Server running on port 3000
```

10. RESULT

Application-level middleware and route handling were successfully implemented in Express.js, verifying request interception, timestamp logging, and response dispatch.